UNIT - II : FUNCTIONS, STRINGS AND LISTS

I. Functions

* A FUNCTION is a named, reusable block of code that performs a specific task. Advantages: code reusability, modularity, easy debugging and maintenance, less repetition. Types: built-in functions, functions in modules, user defined functions.

Built-in functions (commonly used)

Function	Purpose	Example
print(), input()	Output / input	print('hi')
len(x)	Number of items	len('abc') -> 3
type(x), id(x)	Type / address	type(5) -> int
int(), float(), str()	Type conversion	int('7') -> 7
abs(x), pow(x,y)	Absolute value, power	abs(-4) -> 4
max(), min(), sum()	Largest, smallest, total	sum([1,2,3]) -> 6
round(x, n)	Round to n digits	round(3.14159, 2) -> 3.14
sorted(), reversed()	Sort / reverse	sorted([3,1,2])
range(), enumerate(), zip()	Iteration helpers	range(5)
chr(), ord()	Char <-> ASCII	ord('A') -> 65
divmod(a, b)	Quotient and remainder	divmod(7,2) -> (3,1)
eval(), exec()	Evaluate string code	eval('2+3') -> 5

Commonly used modules

* MODULE = a .py file containing functions, classes, variables that can be imported. Forms: import math; import math as m; from math import sqrt, pi; from math import * (not recommended).

```
import math, random, datetime, os, sys
print(math.sqrt(16), math.pi, math.ceil(4.2), math.floor(4.8))
print(math.factorial(5), math.gcd(12, 18), math.pow(2, 3))
print(random.randint(1, 10), random.random())
print(random.choice(['a', 'b', 'c']))
print(datetime.datetime.now())
```

* Other modules: statistics (mean, median), time (sleep, time), calendar, sys (argv), os, json, re, collections, itertools, numpy, pandas. dir(math) lists contents, help(math.sqrt) shows help.

Function definition and calling

* Syntax: def function_name(parameters): body ... return value. PARAMETERS appear in the definition (formal); ARGUMENTS are the values passed when calling (actual). A function runs only when CALLED. Docstring (first string) documents the function.

```
def add(a, b):
    '''returns sum of two numbers'''
    return a + b

result = add(5, 3)        # function call
print(result)             # 8
print(add.__doc__)
```

return statement and void function

* return sends a value back to the caller and ends the function. A function with NO return (or plain return) is a VOID function - it returns None automatically. A function can return multiple values as a tuple.

```
def greet(name):          # void function
    print("Hello", name)

x = greet("Ravi")         # prints Hello Ravi
print(x)                  # None

def square(n):
    return n * n          # fruitful function
```

Scope and lifetime of variables

* LOCAL variable: created inside a function, exists only while the function runs. GLOBAL variable: defined outside, visible everywhere; to modify a global inside a function use the 'global' keyword. nonlocal is for nested functions. Name lookup follows LEGB rule: Local -> Enclosing -> Global -> Built-in.

```
count = 10                # global

def show():
    count = 5             # local, different variable
    print("inside:", count)    # 5

def change():
    global count
    count = 99            # modifies global

show();  print("outside:", count)   # 10
change(); print(count)              # 99
```

Default parameters

* A parameter can have a default value used when the argument is not supplied. Default parameters must come AFTER non-default ones.

```
def power(base, exp=2):
    return base ** exp

print(power(5))         # 25  (exp = 2)
print(power(2, 5))      # 32
```

Keyword arguments

* Arguments passed as name=value, so the order does not matter. Positional arguments must come before keyword arguments.

```
def student(name, age, city="Hyd"):
    print(name, age, city)

student(age=20, name="Anu")            # Anu 20 Hyd
student("Raj", city="Pune", age=21)    # Raj 21 Pune
```

*args and **kwargs

* *args collects any number of POSITIONAL arguments into a TUPLE. **kwargs collects any number of KEYWORD arguments into a DICTIONARY. Order in definition: normal, *args, default, **kwargs.

```
def total(*args):
    s = 0
    for a in args:
        s += a
    return s
print(total(1, 2, 3, 4))       # 10

def info(**kwargs):
    for k, v in kwargs.items():
        print(k, "=", v)
info(name="Anu", age=20)       # name = Anu / age = 20
```

Command line arguments

* Values passed to the program when it is run from the terminal. In the sys module, sys.argv is a LIST of strings: sys.argv[0] = script name, sys.argv[1:] = arguments. (argparse module is the advanced alternative.)

```
# file: add.py      run:  python add.py 10 20
import sys
print("Script:", sys.argv[0])
print("Count:", len(sys.argv))
total = int(sys.argv[1]) + int(sys.argv[2])
print("Sum =", total)            # Sum = 30
```

Recursion and lambda functions

* RECURSION: a function that calls itself; must have a BASE CASE to stop. Default recursion limit is about 1000 (sys.getrecursionlimit()). LAMBDA: small anonymous one-line function: lambda arguments: expression; often used with map(), filter(), sorted().

```
def fact(n):
    if n <= 1:              # base case
        return 1
    return n * fact(n - 1)
print(fact(5))              # 120

sq = lambda x: x * x
print(sq(6))                                    # 36
print(list(map(lambda x: x*2, [1, 2, 3])))        # [2, 4, 6]
print(list(filter(lambda x: x%2==0, range(10))))  # [0, 2, 4, 6, 8]
```

Pass by	Behaviour in Python
Immutable arg (int, str, tuple)	Changes inside function do NOT affect the caller
Mutable arg (list, dict, set)	Changes inside function DO affect the caller (reference is passed)

```
def modify(lst):
    lst.append(99)
nums = [1, 2]
modify(nums)
print(nums)                 # [1, 2, 99]
```

2. Strings

Creating and storing strings

* A string is an immutable sequence of characters (Unicode) in single, double or triple quotes. Stored in a variable; triple quotes allow

multi-line. Escape sequences: \n newline, \t tab, \\ backslash, \' \". Raw string r'C:\new' ignores escapes.

```
s1 = 'Python'
s2 = "Programming"
s3 = '''Multi
line'''
print(len(s1), type(s1))      # 6 <class 'str'>
```

Basic string operations

```
a, b = "Hello", "World"
print(a + " " + b)       # Hello World   (concatenation)
print(a * 3)             # HelloHelloHello  (repetition)
print('e' in a)          # True          (membership)
print(a == "Hello", a < b)   # True True (comparison)
for ch in a: print(ch, end="-")   # H-e-l-l-o-
```

Accessing characters by index

* Index starts at 0 from the left; negative index starts at -1 from the right. Strings are IMMUTABLE: s[0] = 'J' gives TypeError.

```
s = "PYTHON"
#    P  Y  T  H  O  N
#    0  1  2  3  4  5
#   -6 -5 -4 -3 -2 -1
print(s[0], s[3], s[-1])      # P H N
```

String slicing and joining

* Slice s[start:stop:step] - start included, stop EXCLUDED, default step 1; negative step reverses. join() joins elements of an iterable using the string as separator.

```
s = "PYTHON"
print(s[1:4])      # YTH
print(s[:3])       # PYT
print(s[2:])       # THON
print(s[::2])      # PTO
print(s[::-1])     # NOHTYP   (reverse)
print("-".join(["a", "b", "c"]))   # a-b-c
print(" ".join(["I", "love", "Python"]))
```

String methods

Method	Meaning	Example -> result
upper(), lower()	Change case	'abc'.upper() -> 'ABC'
title(), capitalize()	Title / first capital	'hi all'.title() -> 'Hi All'
swapcase()	Swap case	'aB'.swapcase() -> 'Ab'
strip(), lstrip(), rstrip()	Remove spaces	' hi '.strip() -> 'hi'
find(s), index(s)	Position (-1 / error)	'hello'.find('l') -> 2
count(s)	Occurrences	'banana'.count('a') -> 3
replace(old, new)	Replace	'aab'.replace('a','x') -> 'xxb'
split(sep)	String -> list	'a b c'.split() -> ['a','b','c']
startswith(), endswith()	Test prefix / suffix	'file.py'.endswith('.py') -> True
isalpha(), isdigit(), isalnum()	Character tests	'123'.isdigit() -> True
islower(), isupper(), isspace()	Case / space tests	'ab'.islower() -> True
center(w), ljust(w), rjust(w)	Alignment	'hi'.center(6,'*') -> '**hi**'
zfill(w)	Pad with zeros	'7'.zfill(3) -> '007'

```
s = "  Python is fun  "
print(s.strip().upper())          # PYTHON IS FUN
print(s.split())                  # ['Python', 'is', 'fun']
print("banana".count("an"))       # 2
print("hello".replace("l", "L"))  # heLLo
print("madam" == "madam"[::-1])   # True  (palindrome check)
print("a,b,c".split(","))         # ['a', 'b', 'c']
print("Hello".center(11, "*"))    # ***Hello***
```

Formatting strings

```
name, mark = "Anu", 87.456
print("Name: %s, Marks: %.1f" % (name, mark))     # % operator
print("Name: {}, Marks: {:.2f}".format(name, mark))   # format()
print("{1} scored {0}".format(mark, name))        # indexes
print(f"{name} scored {mark:.2f}")                # f-string
print(f"{255:b} {255:o} {255:x} {1234567:,}")     # 11111111 377 ff 1,23
print(f"{name:>10}|{name:<10}|{name:^10}|")       # align
```

3. Lists

Creating lists

* A LIST is an ordered, MUTABLE collection of items (can be of different types), written in square brackets; allows duplicates. Created by [], list(), or list comprehension.

```
a = [10, 20, 30]
b = ["Ram", 25, 5.8, True]       # mixed types
c = list("abc")                  # ['a', 'b', 'c']
d = list(range(1, 6))            # [1, 2, 3, 4, 5]
e = [x * x for x in range(5)]    # [0, 1, 4, 9, 16]
nested = [[1, 2], [3, 4]]        # 2-D list; nested[1][0] = 3
```

Basic list operations

```
x = [1, 2, 3]; y = [4, 5]
print(x + y)          # [1, 2, 3, 4, 5]  concatenation
print(x * 2)          # [1, 2, 3, 1, 2, 3] repetition
print(2 in x)         # True
print(len(x))         # 3
for i in x: print(i, end=" ")
x[0] = 100            # lists are mutable
```

Indexing and slicing in lists

```
L = [10, 20, 30, 40, 50]
print(L[0], L[-1])       # 10 50
print(L[1:4])            # [20, 30, 40]
print(L[:2], L[3:])      # [10, 20] [40, 50]
print(L[::-1])           # [50, 40, 30, 20, 10]
L[1:3] = [21, 31]        # slice assignment
```

Built-in functions used on lists

```
L = [4, 1, 9, 3]
print(len(L), max(L), min(L), sum(L))   # 4 9 1 17
print(sorted(L))                        # [1, 3, 4, 9]  (new list)
print(list(reversed(L)))                # [3, 9, 1, 4]
print(any(L), all(L))                   # True True
for i, v in enumerate(L): print(i, v)
```

List methods

Method	Action
append(x)	Add x at the end
insert(i, x)	Insert x at index i
extend(list)	Add all items of another list
remove(x)	Delete first occurrence of x
pop(i)	Remove and return item at i (default last)
index(x)	Index of first x
count(x)	Number of x
sort(reverse=False)	Sort in place
reverse()	Reverse in place
copy(), clear()	Shallow copy / empty the list

```
L = [3, 1, 2]
L.append(5); L.insert(1, 9); L.extend([7, 8])
print(L)            # [3, 9, 1, 2, 5, 7, 8]
L.remove(9); print(L.pop())      # 8
L.sort(); print(L)              # [1, 2, 3, 5, 7]
L.reverse(); print(L)           # [7, 5, 3, 2, 1]
```

del statement

```
L = [10, 20, 30, 40, 50]
del L[1]          # [10, 30, 40, 50]
del L[1:3]        # [10, 50]
del L             # deletes the whole list (NameError afterwards)
```

List comprehension, copying and 2-D lists

```
sq  = [x**2 for x in range(1, 6)]              # [1, 4, 9, 16, 25]
ev  = [x for x in range(20) if x % 2 == 0]     # even numbers
a = [1, 2, 3]
b = a             # same list (reference): change in b changes a
c = a.copy()      # new list (shallow copy), also a[:] or list(a)
b.append(4); print(a, c)      # [1, 2, 3, 4] [1, 2, 3]
m = [[1, 2, 3], [4, 5, 6]]
print(m[1][2], len(m), len(m[0]))     # 6 2 3
```

List	String
Mutable - can be changed in place	Immutable - new string created on change
Square brackets [ ]	Quotes ' ' or " "
Items of any type	Characters only

pop() / remove() / del - all delete; pop returns the value, del uses index or slice, remove uses the value. sort() changes the list, sorted() returns a new one.

4. Sample Experiments (7 - 12)

Exp 7: Function with multiple return values

```
def calc(a, b):
    return a + b, a - b, a * b, a / b      # returns a tuple

s, d, p, q = calc(20, 4)
print("Sum:", s, "Diff:", d, "Prod:", p, "Quot:", q)
# Output: Sum: 24 Diff: 16 Prod: 80 Quot: 5.0
```

Exp 8: Function using default arguments

```
def interest(p, r=5, t=1):
    return p * r * t / 100

print(interest(1000))              # 50.0
print(interest(1000, 8))           # 80.0
print(interest(1000, 8, 3))        # 240.0
print(interest(1000, t=2))         # 100.0
```

Exp 9: Length of string without library function

```
s = input("Enter a string: ")
count = 0
for ch in s:
    count += 1
print("Length of the string =", count)
# Input: python -> Length of the string = 6
```

Exp 10: Check if a substring is present

```
s = input("Enter string: ")
sub = input("Enter substring: ")
if sub in s:
    print("Substring found")
else:
    print("Substring not found")
# without 'in': loop  for i in range(len(s)-len(sub)+1):
#                     if s[i:i+len(sub)] == sub: found
```

Exp 11: List operations - addition, insertion, slicing

```
L = [10, 20, 30, 40, 50]
L.append(60)              # addition at end
L.insert(2, 25)           # insertion at index 2
print("After add / insert:", L)
print("L[1:4]  =", L[1:4])
print("L[::2]  =", L[::2])
print("L[-3:]  =", L[-3:])
# [10, 20, 25, 30, 40, 50, 60] -> [20, 25, 30] ...
```

Exp 12: Five built-in functions on a list

```
L = [45, 12, 78, 33, 5]
print("len    :", len(L))          # 5
print("max    :", max(L))          # 78
print("min    :", min(L))          # 5
print("sum    :", sum(L))          # 173
print("sorted :", sorted(L))       # [5, 12, 33, 45, 78]
```

Important questions

* 1. Explain default, keyword, *args and **kwargs arguments with examples.

* 2. Scope and lifetime of variables (local, global, LEGB).
* 3. List any 8 string methods with examples; explain string slicing.
* 4. Differentiate list and string; explain list methods with examples.
* 5. What are command line arguments? How are they used in Python?

Revision: def / return / None; LEGB scope; *args (tuple), **kwargs (dict); sys.argv; strings immutable, slicing [start:stop:step]; lists mutable - append, insert, extend, remove, pop, sort, del.